

Sotware ALBA

Documentazione tecnica rielaborata

11 giugno 2026

Indice

1	Introduzione	2
2	Architettura generale del firmware	3
2.1	Periferiche coinvolte	3
2.2	Macchina a stati dell'applicazione	3
3	Inizializzazione del sistema	5
3.1	Inizializzazione BLE	5
3.2	Inizializzazione dei sensori	5
4	Driver della IMU esterna	6
4.1	Indirizzo e registri principali	6
4.2	Configurazione della IMU	6
4.3	Lettura dei dati accelerometrici	7
4.4	Lettura dei dati giroscopici	7
5	Driver del magnetometro	8
5.1	Indirizzo e registri principali	8
5.2	Configurazione del magnetometro	8
5.3	Lettura dei dati magnetici	9
6	Base temporale e gestione del campionamento	10
6.1	Avvio dello streaming	10
6.2	Elaborazione nel main loop	10
6.3	Arresto dello streaming	10
7	Formato dei pacchetti BLE	11
7.1	Ruolo del modulo RN4871	11
7.2	Costruzione del pacchetto	11
7.3	Tipi di dato disponibili	12
8	Flusso operativo completo	13
8.1	Sequenza di esecuzione	13
8.2	Ricostruzione lato app	13
9	Magnetometro e possibili estensioni	15
10	Conclusioni	16
A	Riepilogo rapido per revisione del codice	17
B	References	18

Capitolo 1

Introduzione

Questa documentazione descrive il funzionamento della parte firmware dedicata all'acquisizione dei dati dalla IMU esterna e dal magnetometro della probe, e alla successiva trasmissione dei campioni verso il telefono tramite collegamento BLE. L'obiettivo non è descrivere l'intera applicazione mobile, né tutta la configurazione generata automaticamente da STM32CubeMX, ma isolare il percorso logico che porta dal dato fisico misurato dal sensore al pacchetto ricevuto dall'app.

Nel firmware considerato, il microcontrollore STM32 comunica con i sensori tramite bus I2C e con il modulo BLE RN4871 tramite UART. Il modulo RN4871 si occupa poi della comunicazione wireless verso il telefono. Di conseguenza, dal punto di vista del firmware STM32, l'invio BLE viene trattato come una trasmissione seriale UART: il microcontrollore costruisce un pacchetto di byte e lo invia al modulo, che lo inoltra tramite il servizio BLE configurato.

La catena funzionale può essere riassunta nel modo seguente:

$$\text{sensore} \rightarrow \text{I2C3} \rightarrow \text{STM32} \rightarrow \text{UART3} \rightarrow \text{RN4871} \rightarrow \text{telefono}.$$

La parte centrale da comprendere è che i driver dei sensori leggono registri interni tramite I2C, ricostruiscono i valori raw sugli assi X , Y e Z , mentre il firmware principale invia via BLE i buffer raw da 6 byte. Le conversioni in valori fisici sono disponibili nei driver, ma nella trasmissione BLE corrente vengono usati direttamente i dati grezzi.

Capitolo 2

Architettura generale del firmware

2.1 Periferiche coinvolte

Il firmware utilizza tre blocchi principali: il bus I2C per i sensori, una UART per il modulo BLE e un timer per scandire la frequenza di campionamento. La IMU esterna e il magnetometro sono collegati al bus `I2C3`, gestito tramite l’handle HAL `hi2c3`. Il modulo BLE RN4871 è invece gestito tramite `UART3`, cioè attraverso l’handle `huart3`. Infine, `TIM2` viene usato come base temporale per generare periodicamente la richiesta di un nuovo campione.

La logica è volutamente separata. Il timer non legge direttamente i sensori e non invia direttamente dati BLE. La callback del timer si limita ad alzare un flag, mentre il ciclo principale esegue le operazioni più lente, cioè accessi I2C e trasmissioni UART. Questa scelta è corretta dal punto di vista embedded, perché evita di appesantire la routine di interrupt.

Tabella 2.1: Periferiche principali usate nel flusso di acquisizione e trasmissione.

Elemento	Handle / interfaccia	Funzione nel firmware
IMU esterna	<code>hi2c3</code>	Lettura di accelerometro e giroscopio tramite registri I2C.
Magnetometro	<code>hi2c3</code>	Lettura del campo magnetico sugli assi <i>X</i> , <i>Y</i> e <i>Z</i> .
Modulo BLE RN4871	<code>huart3</code>	Trasmissione dei pacchetti verso il telefono tramite UART trasparente.
Timer di campionamento	<code>htim2</code>	Generazione periodica del flag <code>sample_ready</code> .

2.2 Macchina a stati dell’applicazione

Nel file `main.h` è definita una macchina a stati tramite il tipo `AppState`. Per il comportamento attuale della probe sono rilevanti soprattutto due stati: `STATE_WAIT_BLE_COMMAND` e `STATE_STREAMING`. Il primo rappresenta la condizione in cui la scheda è inizializzata ma non sta inviando campioni; il secondo rappresenta la condizione in cui il timer è attivo e il firmware trasmette periodicamente i dati dei sensori.

Sono presenti anche stati aggiuntivi, come `STATE_ACQUISITION`, `STATE_USB_CONNECTED` e `STATE_DOWNLOAD`, mantenuti per possibili sviluppi futuri legati a data logging, salvataggio su memoria NAND o download tramite USB. Nel flusso BLE attuale questi stati non sono però centrali.

La variabile `streaming_enabled` indica se lo streaming deve essere attivo, mentre `sample_ready` viene usata per segnalare che è arrivato il momento di leggere un nuovo campione. La variabile `ble_rx_byte` contiene invece l'ultimo byte ricevuto dal modulo BLE via UART, usato per interpretare eventuali comandi provenienti dall'app.

Capitolo 3

Inizializzazione del sistema

3.1 Inizializzazione BLE

All'avvio, nel blocco utente del `main.c`, il firmware attende una breve fase di stabilizzazione, accende il LED rosso e inizializza il modulo BLE tramite la funzione `BLE_Initialize()`. Subito dopo abilita la ricezione UART in interrupt con `HAL_UART_Receive_IT()`, in modo da poter ricevere comandi dall'applicazione mobile.

I comandi previsti sono definiti tramite due costanti:

$S \rightarrow$ start streaming, $P \rightarrow$ stop streaming.

Nel codice attuale, tuttavia, lo streaming viene avviato automaticamente al termine dell'inizializzazione tramite `Start_Streaming()`. Questo evita di dipendere dall'affidabilità del comando iniziale inviato dall'app e fa sì che la scheda inizi subito a trasmettere pacchetti BLE.

3.2 Inizializzazione dei sensori

Dopo l'inizializzazione BLE, il firmware verifica la presenza dei sensori. Per la IMU esterna viene chiamata la funzione `EXT_IMU_Init()`, che legge il registro `WHO_AM_I` tramite I2C. Se il valore letto coincide con quello atteso, viene chiamata `EXT_IMU_Config()` per configurare accelerometro e giroscopio.

Lo stesso schema viene usato per il magnetometro. La funzione `MAGN_Init()` legge il registro `WHO_AM_I`; se il dispositivo risponde con il valore corretto, viene eseguita la configurazione tramite `MAGN_Config()`.

Questa struttura ha un vantaggio pratico: prima di configurare un sensore, il firmware verifica che il dispositivo sia effettivamente raggiungibile all'indirizzo I2C previsto. Se il controllo fallisce, viene acceso il LED rosso, segnalando un'anomalia di inizializzazione.

Tabella 3.1: Controllo di presenza dei sensori tramite registro `WHO_AM_I`.

Sensore	Indirizzo I2C	Registro	Valore atteso
IMU esterna	0x6A « 1	0x0F	0x71
Magnetometro	0x1E « 1	0x4F	0x40

Capitolo 4

Driver della IMU esterna

4.1 Indirizzo e registri principali

La IMU esterna è gestita dal driver `ext_imu_driver`. Il driver usa l'handle `hi2c3`, quindi tutte le transazioni verso la IMU passano dal bus I2C3. L'indirizzo del dispositivo è definito come

$$\text{EXT_IMU_I2C_ADDR} = 0\text{x}6\text{A} \ll 1.$$

Lo shift a sinistra è tipico nell'uso delle HAL STM32, perché le funzioni HAL I2C si aspettano l'indirizzo già allineato nel formato a 8 bit, cioè con lo spazio per il bit di lettura/scrittura nella posizione meno significativa.

I registri principali usati dal driver sono riportati in Tabella 4.1.

Tabella 4.1: Registri principali usati per la IMU esterna.

Nome nel driver	Indirizzo	Funzione
EXT_IMU_WHO_AM_I_REG	0x0F	Identificazione del dispositivo.
EXT_IMU_CTRL1_XL	0x10	Configurazione dell'accelerometro.
EXT_IMU_CTRL2_G	0x11	Configurazione del giroscopio.
CTRL3_C	0x12	Configurazione generale, usata per BDU e auto-incremento.
EXT_IMU_OUTX_L_G	0x22	Primo registro dei dati raw del giroscopio.
EXT_IMU_OUTX_L_A	0x28	Primo registro dei dati raw dell'accelerometro.

4.2 Configurazione della IMU

La funzione `EXT_IMU_Config()` scrive tre registri. Il primo è il registro di controllo generale all'indirizzo `0x12`, configurato con il valore `0x44`. Nel commento del codice questo valore è associato all'abilitazione di *Block Data Update* e dell'auto-incremento degli indirizzi. Il Block Data Update è utile perché evita che i byte alto e basso di uno stesso asse vengano aggiornati in momenti diversi durante una lettura multi-byte.

Successivamente vengono configurati accelerometro e giroscopio. Il valore `0x05` viene scritto sia in `CTRL1_XL` sia in `CTRL2_G`. Nel codice questo è commentato come configurazione in high-performance mode a 60 Hz. In questo modo accelerometro e giroscopio vengono abilitati e producono dati periodici leggibili dai rispettivi registri di uscita.

4.3 Lettura dei dati accelerometrici

La funzione `EXT_IMU_ReadAccelerometerData()` legge 6 byte consecutivi a partire dal registro `EXT_IMU_OUTX_L_A`. La lettura avviene tramite `HAL_I2C_Mem_Read()`, specificando indirizzo del dispositivo, indirizzo del registro iniziale, dimensione dell'indirizzo di memoria e buffer di destinazione.

I 6 byte letti rappresentano i tre assi in formato little-endian:

Byte	Contenuto
0	X_{LSB}
1	X_{MSB}
2	Y_{LSB}
3	Y_{MSB}
4	Z_{LSB}
5	Z_{MSB}

Il valore raw di ciascun asse viene ricostruito combinando byte alto e byte basso:

$$\text{raw}_x = (X_{MSB} \ll 8) \mid X_{LSB}.$$

Nel codice questa operazione viene poi castata a `int16_t`, perché i dati del sensore sono signed a 16 bit. Dopo la ricostruzione del raw, il driver calcola anche il valore convertito moltiplicando per la sensibilità accelerometrica:

$$a_x = \text{raw}_x \cdot 0.061.$$

La stessa operazione viene ripetuta per gli assi Y e Z .

4.4 Lettura dei dati giroscopici

La funzione `EXT_IMU_ReadGyroscopeData()` segue esattamente la stessa logica, ma legge a partire dal registro `EXT_IMU_OUTX_L_G`. Anche in questo caso vengono acquisiti 6 byte e ricostruiti tre valori signed a 16 bit.

La conversione interna al driver usa la sensibilità giroscopica:

$$\omega_x = \text{raw}_x \cdot 8.75.$$

Anche se queste conversioni sono disponibili nelle strutture `EXT_IMU_Data`, il flusso BLE corrente non invia direttamente i valori floating point. Il pacchetto BLE riceve invece il buffer raw, cioè i 6 byte letti dal sensore.

Nel firmware attuale la IMU esterna è il sensore principale usato nello streaming realtime. A ogni campione vengono letti accelerometro e giroscopio, e vengono inviati due pacchetti BLE distinti: uno di tipo accelerometro esterno e uno di tipo giroscopio esterno.

Capitolo 5

Driver del magnetometro

5.1 Indirizzo e registri principali

Il magnetometro è gestito dal driver `magn_driver`. Anche questo dispositivo usa il bus I2C3, sempre tramite l'handle `hi2c3`. L'indirizzo del magnetometro è definito come

$$\text{MAGN_I2C_ADDR} = 0x1E \ll 1.$$

La presenza del dispositivo viene verificata leggendo il registro `WHO_AM_I` all'indirizzo `0x4F`. Il valore atteso è `0x40`. Se il confronto ha esito positivo, il firmware considera il magnetometro correttamente rilevato.

Tabella 5.1: Registri principali usati per il magnetometro.

Nome nel driver	Indirizzo	Funzione
<code>MAGN_WHO_AM_I_REG</code>	<code>0x4F</code>	Identificazione del dispositivo.
<code>MAGN_CFG_REG_A</code>	<code>0x60</code>	Configurazione della modalità operativa.
<code>MAGN_CFG_REG_B</code>	<code>0x61</code>	Configurazione della cancellazione offset.
<code>MAGN_CFG_REG_C</code>	<code>0x62</code>	Configurazione aggiuntiva, tra cui BDU.
<code>MAGN_STATUS_REG</code>	<code>0x67</code>	Registro di stato, disponibile per controlli futuri.
<code>MAGN_OUTX_L_REG</code>	<code>0x68</code>	Primo registro dei dati magnetici raw.

5.2 Configurazione del magnetometro

La funzione `MAGN_Config()` scrive tre registri di configurazione. Il valore `0x00` viene scritto in `CFG_REG_A` e nel commento del codice corrisponde alla modalità continua a 10 Hz. Il valore `0x02`, scritto in `CFG_REG_B`, abilita la cancellazione dell'offset. Infine, il valore `0x10`, scritto in `CFG_REG_C`, abilita il Block Data Update.

Questa configurazione rende il magnetometro pronto a produrre misure periodiche del campo magnetico. L'uso del Block Data Update è utile anche qui, perché riduce il rischio di leggere una parte del campione precedente e una parte del campione successivo durante una lettura multi-byte.

5.3 Lettura dei dati magnetici

La funzione `MAGN_ReadMagnetometerData()` legge 6 byte consecutivi a partire dal registro `MAGN_OUTX_L_REG`. La struttura dei dati è analoga a quella della IMU:

$$X_{LSB}, X_{MSB}, Y_{LSB}, Y_{MSB}, Z_{LSB}, Z_{MSB}.$$

Ogni asse viene ricostruito come intero signed a 16 bit. Il driver converte poi il valore raw in mGauss usando la sensibilità

$$S_m = 1.5 \text{ mGauss/LSB}.$$

Quindi, per esempio, il valore sull'asse X viene calcolato come

$$B_x = \text{raw}_x \cdot 1.5.$$

Nel firmware corrente il magnetometro viene inizializzato e configurato, ma la lettura e la trasmissione BLE sono commentate all'interno della funzione `Process_Sensor_Sample()`. Questo significa che il magnetometro è predisposto per sviluppi futuri, ad esempio sensor fusion o orientamento rispetto al campo magnetico terrestre, ma non partecipa allo streaming realtime attivo.

Capitolo 6

Base temporale e gestione del campionamento

6.1 Avvio dello streaming

La funzione `Start_Streaming()` prepara lo streaming realtime. Prima azzerava il flag `sample_ready` e il divisore del magnetometro `magn_divider`; poi imposta `streaming_enabled = 1` e porta la macchina a stati in `STATE_STREAMING`. Successivamente accende il LED verde e avvia il timer TIM2 in modalità interrupt tramite `HAL_TIM_Base_Start_IT()`.

Il timer viene quindi usato come sorgente periodica di eventi. Ogni volta che il timer scade, viene eseguita la callback `HAL_TIM_PeriodElapsedCallback()`. Nel caso in cui l'handle ricevuto sia `htim2`, la callback non legge sensori e non invia dati, ma imposta soltanto:

```
sample_ready = 1.
```

6.2 Elaborazione nel main loop

Nel ciclo principale, quando lo stato corrente è `STATE_STREAMING`, il firmware controlla se `sample_ready` è alto. Se lo è, il flag viene immediatamente riportato a zero e viene chiamata la funzione `Process_Sensor_Sample()`.

Questa struttura separa il tempo di campionamento dalla lettura effettiva. Il timer dà il ritmo, mentre il main loop esegue le operazioni I2C e UART. Il vantaggio è che la callback del timer resta molto breve e deterministica. In un sistema embedded questo è importante, perché operazioni come `HAL_I2C_Mem_Read()` o `HAL_UART_Transmit()` possono richiedere un tempo non trascurabile e non dovrebbero essere eseguite direttamente dentro un interrupt periodico, se non strettamente necessario.

6.3 Arresto dello streaming

La funzione `Stop_Streaming()` disabilita lo streaming riportando `streaming_enabled` a zero, azzerava `sample_ready`, ferma TIM2 con `HAL_TIM_Base_Stop_IT()` e riporta lo stato a `STATE_WAIT_BLE_COMMAND`. Inoltre spegne il LED verde.

Il comando di stop può arrivare dall'app tramite UART. Quando viene ricevuto il byte P, la callback `HAL_UART_RxCpltCallback()` pone `streaming_enabled = 0`. Il main loop rileva questo valore e chiama `Stop_Streaming()`.

Capitolo 7

Formato dei pacchetti BLE

7.1 Ruolo del modulo RN4871

Il modulo RN4871 viene usato come interfaccia BLE esterna. Dal punto di vista del microcontrollore, esso è controllato tramite UART. La funzione `BLE_SendData()` invia un buffer di byte usando `HAL_UART_Transmit()` sull'handle `huart3`. In altre parole, il firmware non costruisce direttamente pacchetti BLE a basso livello: costruisce pacchetti applicativi e li invia via UART al modulo BLE.

Questo approccio semplifica il firmware, perché la gestione della connessione BLE, dell'advertising e del profilo Transparent UART è delegata al modulo RN4871. Per la parte applicativa è sufficiente rispettare un formato di pacchetto coerente tra firmware e app.

7.2 Costruzione del pacchetto

La funzione `BLE_SendPacket()` costruisce un pacchetto di lunghezza fissa pari a `PACKET_LENGTH`, definita nel file `bluetooth.h` come 20 byte. Il primo byte è il delimitatore di inizio pacchetto, mentre l'ultimo byte è il delimitatore di fine pacchetto:

$$\text{byte } 0 = \{, \quad \text{byte } 19 = \}.$$

I byte intermedi vengono inizializzati a zero. Il byte 1 identifica il tipo di dato trasmesso. Per la IMU esterna vengono usati due caratteri distinti: **a** per l'accelerometro esterno e **g** per il giroscopio esterno. Il carattere **M** è previsto per il magnetometro, anche se nel flusso attuale la sua trasmissione è disabilitata.

I byte da 2 a 7 contengono il buffer raw del sensore:

Byte pacchetto	Contenuto
2	X_{LSB}
3	X_{MSB}
4	Y_{LSB}
5	Y_{MSB}
6	Z_{LSB}
7	Z_{MSB}

I byte da 8 a 18 restano a zero nella versione corrente. La struttura completa è quindi riportata in Tabella 7.1.

Tabella 7.1: Formato del pacchetto BLE applicativo.

Byte	Campo	Descrizione
0	Start delimiter	Carattere { per indicare l'inizio del pacchetto.
1	Data type	Identifica il tipo di dato: a, g, M, ecc.
2-3	X raw	Byte basso e byte alto dell'asse X.
4-5	Y raw	Byte basso e byte alto dell'asse Y.
6-7	Z raw	Byte basso e byte alto dell'asse Z.
8-18	Padding	Byte inizializzati a zero, disponibili per estensioni future.
19	End delimiter	Carattere } per indicare la fine del pacchetto.

7.3 Tipi di dato disponibili

L'enumerazione `BLE_DataType` definisce i possibili tipi di dati inviabili. Nel file `bluetooth.c`, questi tipi vengono convertiti in caratteri identificativi all'interno del pacchetto.

Tabella 7.2: Associazione tra tipo dati firmware e identificatore nel pacchetto.

Tipo firmware	ID pacchetto	Significato
<code>DATA_TYPE_IMU_ACCELERATION</code>	A	Accelerometro IMU interna.
<code>DATA_TYPE_IMU_GYROSCOPE</code>	G	Giroscopio IMU interna.
<code>DATA_TYPE_EXT_IMU_ACCELERATION</code>	a	Accelerometro IMU esterna.
<code>DATA_TYPE_EXT_IMU_GYROSCOPE</code>	g	Giroscopio IMU esterna.
<code>DATA_TYPE_MAGNETOMETER</code>	M	Magnetometro esterno.

A ogni campione attivo, il firmware invia due pacchetti consecutivi: prima il pacchetto dell'accelerometro esterno, poi quello del giroscopio esterno. Entrambi contengono 6 byte raw, non i valori floating point già convertiti dal driver.

Capitolo 8

Flusso operativo completo

8.1 Sequenza di esecuzione

Mettendo insieme le parti precedenti, il comportamento della versione corrente del firmware può essere descritto come una sequenza ordinata. Prima vengono inizializzate le periferiche generate da CubeMX; poi, nei blocchi utente, viene inizializzato il modulo BLE, viene abilitata la ricezione UART in interrupt e vengono verificati i sensori tramite registri `WHO_AM_I`. Se la IMU esterna e il magnetometro rispondono correttamente, vengono configurati.

Successivamente il firmware avvia automaticamente lo streaming. Il timer TIM2 genera periodicamente un evento di update; la callback del timer imposta `sample_ready`; il main loop vede il flag e chiama `Process_Sensor_Sample()`. Questa funzione legge accelerometro e giroscopio della IMU esterna, ottenendo due buffer raw da 6 byte. Infine, ciascun buffer viene passato a `BLE_SendPacket()`, che costruisce un pacchetto da 20 byte e lo invia via UART al modulo RN4871.

Il telefono riceve quindi pacchetti distinti per accelerazione e velocità angolare. L'app deve riconoscere i delimitatori { e }, leggere il byte di tipo e ricostruire i tre assi combinando LSB e MSB nello stesso ordine usato dal firmware.

8.2 Ricostruzione lato app

Poiché il firmware invia i dati raw, l'applicazione deve ricostruire gli interi signed a 16 bit. Per l'asse X , usando i byte 2 e 3 del pacchetto, la ricostruzione è concettualmente:

$$\text{raw}_x = (\text{byte}_3 \ll 8) \mid \text{byte}_2.$$

Lo stesso vale per Y e Z :

$$\text{raw}_y = (\text{byte}_5 \ll 8) \mid \text{byte}_4,$$

$$\text{raw}_z = (\text{byte}_7 \ll 8) \mid \text{byte}_6.$$

Dopo la ricostruzione, l'app può applicare le sensibilità se vuole visualizzare valori fisici. Per l'accelerometro esterno si usa la sensibilità impostata nel driver, cioè 0.061; per il giroscopio esterno si usa 8.75. Per il magnetometro, quando verrà riabilitato, la sensibilità prevista è 1.5 mGauss/LSB.

Tabella 8.1: Sensibilità usate dai driver per la conversione dei dati raw.

Misura	Sensibilità	Nota
Accelerometro esterno	0.061	Valore moltiplicato per il raw a 16 bit.
Giroscopio esterno	8.75	Valore moltiplicato per il raw a 16 bit.
Magnetometro	1.5 mGauss/LSB	Usato nel driver, ma trasmissione attualmente disabilitata.

Capitolo 9

Magnetometro e possibili estensioni

Il codice del magnetometro è già presente e funzionante a livello di driver. La parte disabilitata si trova nel flusso di streaming, dove la lettura `MAGN_ReadMagnetometerData()` e l'invio `BLE_SendPacket(DATA_TYPE_MAGNETOMETER, raw_magnetometer)` sono commentati.

La scelta è coerente con i commenti del firmware: il magnetometro viene tenuto disponibile per sviluppi futuri, ma non viene trasmesso nella versione attuale per ridurre il traffico BLE e migliorare la stabilità della connessione durante lo streaming. Essendo la IMU configurata a 60 Hz e il magnetometro a 10 Hz, non avrebbe necessariamente senso inviare il magnetometro alla stessa frequenza di accelerometro e giroscopio.

Per questo motivo è stato previsto il divisore `MAGN_SEND_DIVIDER`, pari a 5. L'idea è incrementare `magn_divider` a ogni campione e inviare il magnetometro solo quando il divisore raggiunge la soglia. In questo modo, se la IMU viene campionata più velocemente del magnetometro, il dato magnetico può essere inviato a frequenza ridotta, riducendo il carico sulla comunicazione BLE.

Per riabilitare il magnetometro nello streaming non basta avere il driver: bisogna decommentare la sezione in `Process_Sensor_Sample()`, verificare che la frequenza di invio sia compatibile con il throughput BLE e aggiornare l'app per interpretare i pacchetti con identificatore M.

Capitolo 10

Conclusioni

Il firmware implementa una catena di acquisizione semplice ma ordinata. La IMU esterna e il magnetometro sono inizializzati tramite I2C leggendo i rispettivi registri di identificazione. La IMU esterna viene configurata per produrre dati di accelerometro e giroscopio, che vengono letti periodicamente a partire dai registri di uscita. Ogni lettura acquisisce 6 byte, corrispondenti agli assi X , Y e Z in formato little-endian.

La temporizzazione dello streaming è gestita da un timer, ma le operazioni lente vengono eseguite nel main loop attraverso un flag. Questa separazione rende il firmware più robusto e leggibile. La trasmissione BLE avviene tramite il modulo RN4871, controllato via UART3. Il firmware costruisce pacchetti applicativi da 20 byte, contenenti delimitatore iniziale, tipo del dato, 6 byte raw del sensore e delimitatore finale.

Nella versione corrente sono effettivamente trasmessi accelerometro e giroscopio della IMU esterna. Il magnetometro è inizializzato e configurato, ma la sua lettura e trasmissione sono lasciate come funzionalità futura, probabilmente da riattivare quando sarà necessario integrare orientamento magnetico o sensor fusion.

Appendice A

Riepilogo rapido per revisione del codice

Tabella A.1: Funzioni principali coinvolte nel flusso sensori-BLE.

Funzione	Ruolo
EXT_IMU_Init()	Verifica la presenza della IMU esterna tramite registro WHO_AM_I.
EXT_IMU_Config()	Configura accelerometro, giroscopio, BDU e auto-incremento.
EXT_IMU_ReadAccelerometerData()	Legge 6 byte raw dell'accelerometro e calcola anche i valori convertiti.
EXT_IMU_ReadGyroscopeData()	Legge 6 byte raw del giroscopio e calcola anche i valori convertiti.
MAGN_Init()	Verifica la presenza del magnetometro tramite registro WHO_AM_I.
MAGN_Config()	Configura modalità continua, offset cancellation e BDU del magnetometro.
MAGN_ReadMagnetometerData()	Legge 6 byte raw del magnetometro e calcola i valori in mGauss.
Start_Streaming()	Abilita lo streaming e avvia TIM2 in interrupt.
Process_Sensor_Sample()	Legge i dati della IMU esterna e li invia tramite BLE_SendPacket().
BLE_SendPacket()	Costruisce il pacchetto da 20 byte e lo invia al modulo BLE.

Appendice B

References

Bibliografia

- [1] Codice firmware fornito per acquisizione dati da IMU esterna, magnetometro e trasmissione BLE tramite RN4871.
- [2] Template LaTeX fornito come riferimento per struttura, pacchetti e stile della documentazione.